

ORIGINAL RESEARCH

Graph neural networks as strategic transport modelling alternative - A proof of concept for a surrogate

Santhanakrishnan Narayanan  | Nikita Makarov | Constantinos Antoniou 

TUM School of Engineering and Design, Chair of Transportation Systems Engineering, Technical University of Munich, Munich, Germany

Correspondence

Santhanakrishnan Narayanan, TUM School of Engineering and Design, Chair of Transportation Systems Engineering, Technical University of Munich, Arcisstrasse 21, 80333 Munich, Germany. Email: santhanakrishnan.narayanan@tum.de

Funding information

HORIZON EUROPE Framework Programme, Grant/Award Number: 101076963; Deutsche Forschungsgemeinschaft, Grant/Award Number: GSC 81

Abstract

Practical applications of graph neural networks (GNNs) in transportation are still a niche field. There exists a significant overlap between the potential of GNNs and the issues in strategic transport modelling. However, it is not clear whether GNN surrogates can overcome (some of) the prevalent issues. Investigation of such a surrogate will show their advantages and the disadvantages, especially throwing light on their potential to replace complex transport modelling approaches in the future, such as the agent-based models. In this direction, as a pioneer work, this paper studies the plausibility of developing a GNN surrogate for the classical four-step approach, one of the established strategic transport modelling approaches. A formal definition of the surrogate is presented, and an augmented data generation procedure is introduced. The network of the Greater Munich metropolitan region is used for the necessary data generation. The experimental results show that GNNs have the potential to act as transport planning surrogates and the deeper GNNs perform better than their shallow counterparts. Nevertheless, as expected, they suffer performance degradation with an increase in network size. Future research should dive deeper into formulating new GNN approaches, which are able to generalize to arbitrary large networks.

1 | INTRODUCTION

Transportation plays a critical role in modern society. For example, commute time is shown to be the strongest factor affecting whether a family can escape poverty [1], showing that it is essential that transportation projects be planned out well. Focusing on long term travel modelling, various tools have been developed over the decades, with the four step model being the most popular approach [2]. This method has a number of known drawbacks with strong assumptions on the underlying decision-making process. Another drawback is that the four step model outputs point values, not statistical distributions of the modelled measures. Due to all models having a certain degree of error to them, it is thus harder for decision-makers to trust the model's output [3]. Various advancements have been made in the field of transport modelling. Agent-based approaches are becoming popular and are regarded as appropriate to model emerging solutions [4]. Nevertheless, such approaches are cumbersome and complex for the cities to use them. Furthermore, they require a lengthy and manual calibration process.

Looking at transport systems, new modes are emerging, changing the underlying patterns of how users make choices [5]. Additionally, the popularity of mobile phones and information platforms has widespread effects on travel choices [6]. Since early 2020, COVID-19 has also drastically affected the number of trips people are taking and the modes that they feel safe in [7]. On the one hand, the combination of these effects shows that quickly adaptable transport planning systems are needed for the coming decades. On the other hand, both more data and computational power is becoming available for transport planning, meaning that it is now possible to employ more data driven methods [8]. The main underlying difference of data driven methods, in comparison to model based methods, is that the exact model formulation is learnt from the data and the method can thus exploit patterns, which would be overlooked if done otherwise.

In data driven methods, machine learning has become the prevailing class of methods [9]. Thus, there has been a growing popularity of machine learning for transport planning [10]. In the field of machine learning, the deep learning subclass

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2024 The Author(s). *IET Intelligent Transport Systems* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

of methods have been breaking previous baselines in several fields, ranging from image recognition to natural language processing [11]. The main advantage of deep learning methods is their ability to model highly non-linear processes and being universal approximators to any given function. Since 2015, a new class of deep learning methods have appeared which apply deep learning functions on graphs, called the graph neural networks (GNNs) [12]. As transport networks can be easily interpreted as graphs, it is a natural application for GNNs. Naturally, they have been applied for multiple problems in the field of transportation, e.g. short-term traffic forecasting and short-term demand prediction for shared mobility services [13].

Examining simulation sciences, a sub-field called surrogate modelling has been rising, where instead of running simulation functions directly, a second function is built to mimic the original function, the so-called surrogate model [14]. The surrogate model possesses some properties that the original function does not have, such as being computationally cheap. As machine learning models are often faster to apply and can approximate various functions, they have often been employed as surrogates [15]. Due to matrix-based operations, deep learning surrogates are able to perform highly efficiently. In a classification based scenario, deep learning models can also output probability scores for each category while training directly from real, observed data. Specifically, graph neural network surrogates have recently reached state of the art levels in certain fields of physics, such as cloth simulations [16].

Comparing the issues of the existing transport planning methods with the potential of a graph neural network-based surrogate (i.e. uncertainty quantification through confidence scores, the possibility of automatic model calibration, easier model application and low prediction time), it can be seen that there is a significant overlap. Furthermore, in other fields, GNN surrogates are shown to generalize to domains much larger than the training data [17]. Therefore, this study attempts to explore whether a graph neural network surrogate function can be developed for transport planning. As an initial step towards this direction, this paper focuses on the development of a GNN surrogate for the four step transport modelling approach and examine where the limits currently exist.

Thus, the main contributions of this study are two-fold:

- A pioneer work in the formulation of a deep learning GNN surrogate for a strategic transport modelling approach, i.e. the four step model in the current case, which can aid in understanding the potential and limitations of such surrogates for strategic transport planning.
- Insights obtained from the experiments performed to ascertain a feasible surrogate configuration can support future researchers to properly design such frameworks for the problem explored in this study, as well as several other transport planning problems.

These main contributions pave way to a number of complementary contributions. These include (i) the design of augmented data generation process to overcome data limitations, (ii) the development of data transformation procedures

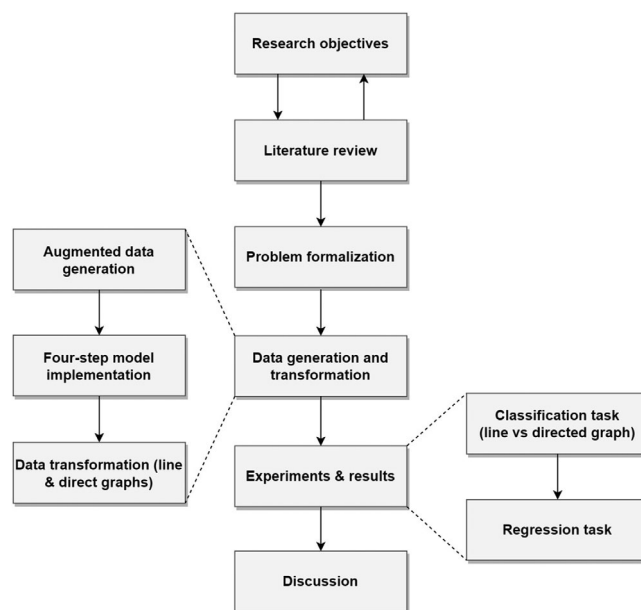


FIGURE 1 Research approach.

and (iii) the framing of appropriate regularization methods, loss functions and evaluation metrics. Besides, the results of this study open up the strategic transport planning problem to deep learning research.

The research approach followed in this study is outlined in Figure 1. Following the specification of the research objectives, a review of the existing literature is performed (Section 2). Subsequently, the problem tackled in this study is defined formally (Section 3). Afterwards, the data generation and transformation processes are described (Section 4). Then, the modelling experiments are performed (Section 5) and the results are shared (Section 6). At last, the conclusions are elucidated (Section 7), along with the directions for future research. The entire procedure is scripted in Python [18].

2 | LITERATURE REVIEW

This section focuses on the existing literature related to (i) the application of machine learning models in the context of the four-step transport modelling approach and transport planning and (ii) surrogate modelling. In the former section, a generic focus on the machine learning is maintained, as there is no substantial research on the application of GNNs in the context of strategic transport modelling approaches, especially the four step modelling approach. Furthermore, we also review studies focusing on different neural network variants (e.g. recurrent neural networks), as they are considered to be the subsets of machine learning [19, 20]. We also maintain this notion throughout this article. The reviewed studies were gathered through the Scopus database using an [open source python script](https://github.com/nsanthanakrishnan/Scopus-Query)¹ from [21].

¹ <https://github.com/nsanthanakrishnan/Scopus-Query>

2.1 | Application of machine learning models in the context of the four-step transport modelling approach

Machine learning methods have the advantage of being able to model highly complex patterns. Especially in recent years, with the availability of both more data and better methods, machine learning has become a popular topic, often replacing previously used analytical and statistical methods. For example, to improve the trip generation process, Saleh et al. [22] apply neural networks for trip production and attraction rates. Xiao et al. [23] introduce a new method combining convolutional neural networks with recurrent neural networks. The authors of these papers find that machine learning approaches outperform the baselines, though sometimes with overfitting problems and challenges in interpretability. Nevertheless, Mukherjee and Raghuram Kadal [24] conclude that machine learning modelling approaches are less explored for the trip generation process. For the trip distribution stage, Pourebrahim et al. [25] use gravity models, random forests and neural networks on Twitter data to enhance the model precision. Goel and Sinha [26] use an adaptive neuro-fuzzy inference system to predict trip distribution in Delhi. Pitombo et al. [27] develop decision tree algorithms to take disaggregate factors into account. Again, these papers find that the machine learning models perform better but have problems with interpretability.

In the existing literature, most machine learning works have been on mode choice. An overview of the application of machine learning for mode choice can be found in [28]. Nevertheless, in this paragraph, some of the methods used are highlighted. Garcia-García et al. [29] examine and compare different machine learning, deep neural networks and random utility maximization models for travel mode choice modelling. Zhao et al. [30] also compare various machine learning approaches, but additionally introduce a new method for computing elasticities, which is of importance for interpretation. Narayanan et al. [31] went a step ahead and compared the relative importance of variables as ascertained by a random forest classifier and a logit model. They concluded that a coherence exists the two approaches, although some discrepancy is also observed. All the aforementioned studies consistently find that random forest approach works best. When it comes to traffic assignment, there has been comparatively little research done, and mostly, the studies focus on the application of reinforcement learning. For example, Grunitzki and Bazzan [32] have developed a reinforcement learning algorithm for the traffic assignment equilibrium. Zhou et al. [33] also use reinforcement learning, but with an updated potential function. These papers find that the reinforcement learning models can outperform the baselines in certain cases, but are highly sensitive to input features. The result of such a model is that it allows the traffic modeller to exploit multiple data sources simultaneously. To enable the computation of the gradients, the authors propose to sample the possible paths taken, meaning that not all possibilities are considered. The method is computationally expensive, growing exponentially with the size of the network.

End to end deep learning has been able to achieve breakthrough results in a number of fields, such as automatic translation and image recognition, by being able to merge multiple modelling steps together into a single large model, such as feature engineering as well as output processing [11, 34]. In the context of transport planning, the closest work that has been found is in the field of short term traffic prediction, where several work has been recently published. The background here is to apply models in the operational setting. The problem setting is to predict the future traffic conditions, given a current traffic state. The time range considered is usually within a few minutes or hours. There are further sub-problems, such as predicting traffic flow, speed and travel time.

An in-depth survey of the application of deep learning for short term traffic prediction is provided in [35]. The recent trend has been towards representing the traffic state as a graph. The usual configuration is to have the nodes representing intersections and edges representing street links. With this approach, it is possible to apply graph neural networks. Focusing on graph neural network literature, Jiang and Luo [36] provide a survey of the existing research on this topic, which majorly focus on short term traffic prediction. There are various graph neural network types, and the survey finds that the graph convolutional network (GCN) is the most popular method. Additionally, some recent papers propose extensions, by adding a recurrent component to model time dependency (e.g. [37]) or implementing adaptive processes for spatial dependency (e.g. [38]). As GCN employ only node information, there has been work examining the usage of converting the input graph first into a line graph [39].

2.2 | Surrogate modelling

Coming from simulation sciences, surrogate modelling is when an expensive function is approximated by a function (i.e. the surrogate model), which possess some positive qualities, such as cheap evaluation costs [14]. Traditionally, this has been done with polynomials, but recently deep learning models have also started to be used as surrogates and have achieved significant results [15]. Shan and Wang [40] provide a summary of classical approaches, which are primarily focused on decomposing the models into linear combinations of simpler functions, such as orthogonal polynomials. As deep learning models are cheap to evaluate and handle high dimensional problems well, today, they are becoming a natural candidate for surrogates [41]. The common design principle is to generate or retrieve a large number of input samples and feed them through the simulation. The resulting simulation results are then used as targets for deep learning models, whilst the inputs stay the same. To the best of authors' knowledge, there is no surrogate model existing in the literature for transport modelling approaches. Therefore, some examples from other fields are reviewed in the subsequent paragraph.

Surrogate modelling is a broad field, and this paragraph provides a summary of literature focusing only on GNN surrogates. Where data are expensive to obtain, deep learning surrogates

can be beneficial for uncertainty quantification [42]. Nevertheless, most existing GNN surrogates are from the physics simulation field. For example, Franco et al. [43] develop surrogate models to replace computationally expensive solvers for the simulation of time-dependent partial differential equations. They state that GNN surrogates can provide a valid alternative to traditional surrogate models in terms of computational efficiency and generalization to new scenarios. They also mention that GNNs are better than the classical dense deep neural networks, from a numerical standpoint. On a different note, Ogoke et al. [44] create a GCN surrogate for aerofoil drag force prediction, where the aerofoil environment is modelled as a mesh. The results show that the model achieves highly accurate results. Choi and Kumar [17], who built a GNN surrogate to simulate granular flow dynamics, conclude that such a surrogate is hundreds of times faster than high-fidelity numerical simulators. Furthermore, they observe that the surrogate generalizes to domains much larger than the training data.

2.3 | Summary & research gaps

In this section, a summary of the reviewed studies and the identified gaps are provided, which act as a motivation for the research objectives of this study. Summarizing the work on machine learning models in relation to strategic transport modelling, it can be seen that the existing studies focus on the individual steps, especially on mode choice. On the one hand, to the best of authors' knowledge, machine learning approaches have not been used to model the steps in a combined manner. On the other hand, graph neural networks are observed to achieve the state of the art performance for short term traffic prediction.

Therefore, an option is to develop a graph neural surrogate for a strategic transport modelling approach, focusing on long term transport planning. Such an option makes sense, as surrogate modelling from other fields has shown that it can relieve many of the problems facing simulations, by using deep learning. Besides, due to the nature of neural networks, they implicitly do not require manual calibration, as their parameters are automatically learned from the data. Some hyperparameter tuning and network selection may be needed, however there exist automated algorithms for this as well [45]. The so called online tuning allows the surrogate to continuously improve itself with automatic data gathering, thus having a model which is always updated to the latest information. Finally, graph neural networks with GPU acceleration have much lower computational time than traditional simulation techniques, with surrogates experiencing a decrease in computation time on the order of one to two magnitudes [16].

3 | PROBLEM FORMALIZATION AND METHODOLOGY

The surrogate model framework is formally defined at first, followed by a presentation of a generic algorithm for its use. Finally, the methodological scope of this paper is described.

3.1 | Surrogate model definition

Modern graph neural networks are universal approximators, meaning that they can model any function within a given error, modulo the limits enforced by the 1-WL test [46]. This implies that, with graph neural networks, any decision making pattern of a traveller can be modelled, as long as it is not completely random. In this paper, the objective is to build a GNN surrogate model, which takes socio-economic data and network information as input, whilst producing traffic predictions. The problem can be formalized as follows:

$$f_{TP} = \arg \min_{f'_{TP}} e(S, G(\mathcal{N}, \mathcal{A}), \mathcal{T}) \in D (f'_{TP}(S, G(\mathcal{N}, \mathcal{A})), \mathcal{T}),$$

$$f_{TP} : S' \times \mathcal{N}' \rightarrow \mathcal{T}' \quad (1)$$

where, S' being the set of socio-economic data for a given region. $\mathcal{N}'$ being the set of networks with their respective properties. $\mathcal{T}'$ being the traffic flow along with the network for each mode. D is the set of real-world, observed data which contains triplets of the socio-economic data (S), network information ($G(\mathcal{N}, \mathcal{A})$) and resulting traffic flow ($\mathcal{T}$). f_{TP} is the optimal transport planning model defined as a function, which is the closest model to the observed data, whilst f'_{TP} is a possibly non-optimal model being evaluated on the observed data. e is an error function over the given dataset, which compares the prediction of the model with the true value. The exact form is defined based on the task at hand, but a standard error function is, for example, the mean squared error [47].

The exact formalization of the datasets depends highly on both the observed datasets as well as the modelling approach used. In practice, the model that is selected should possess some inherent properties, such as being the most simple yet accurate model within the search space. At this stage, it is also important to note the similarity between Equation (1) and the general optimization criteria used in machine learning [48], highlighting that the task comes naturally to machine learning models.

3.2 | GNN surrogate for the four step modelling approach

The aim of surrogate modelling is to find a surrogate $g \in G$ that is as close as possible to the four step model f_{4SM} . f_{4SM} is the best fitting four step model, calibrated to the observed data. G is the set of functions taking s, n as input and producing t as output, which (i) are cheaper to evaluate than the actual transport modelling approach, (ii) can automatically calibrate to data and (iii) allow non-deterministic predictions. Graph neural networks possess these qualities. Thus, the search for the surrogate becomes $g \in G_{GNN}$. Formally,

$$g = \arg \min_{g' \in G_{GNN}} e(S, G(\mathcal{N}, \mathcal{A}), \cdot) \in D (f_{4SM}(S, G(\mathcal{N}, \mathcal{A})), g'(S, G(\mathcal{N}, \mathcal{A}))) \quad (2)$$

where, $\mathcal{D}$ is the set of real-world, observed data which contains triplets of the socio-economic data ($\mathcal{S}$), network information ($G(\mathcal{N}, \mathcal{A})$) and resulting traffic flow ($\mathcal{T}$). g is the optimal surrogate model, whilst g' is a non-optimal model being evaluated on the training data. $e_{(\mathcal{S}, G(\mathcal{N}, \mathcal{A}), \mathcal{T}) \in \mathcal{D}}$ again being the error over the dataset, but this time comparing the outputs of the four step model with that of the surrogate. 4SM is the best fitting four step model, calibrated to the observed data

Examining Equation 2, it can be seen that the surrogate model is compared to the output data from a four-step model to assess its performance. In this paper, augmented data (augmented implies that a true data source is employed, but modified to produce more data) is employed to generate adequate data points (called $D_{\text{augmented}}$) for the development of GNNs, as will be discussed in Section 4.

Finally, note that any machine learning method can be used as the surrogate model. In this case, in Equation (2), the $\mathcal{G}_{\text{GNN}}$ class of methods is replaced by the respective machine learning class. The rest of the equation does not change, and the procedure stays identical. This property is used for developing the baseline models.

The surrogate model could be used in practice through the following four steps:

- At first, a large dataset is generated using existing four step models on cities, to determine the traffic flow, given the socio-economic and network data. Note, this step would have to be done only once.
- Secondly, a graph neural network is trained on the aforementioned dataset under inductive setting. Again, this step would have to be done only once.
- Thirdly, a traffic engineer takes the pre-trained model and applies it to a region of interest to directly predict traffic for certain scenarios. The model takes socio-economic and network data as input, whilst producing traffic flow predictions for the entire region as output.
- Finally, as data is automatically gathered in a specific region, the model is automatically calibrated to fit the new data, thus always being up to date with the current local transport patterns.

The initial two steps are performed to make the GNN surrogate mimic the actual strategic transport model and learn the relevant patterns. This initial training requires larger datasets. However, once this has been carried out, new datasets (with smaller sized samples) could be utilized to fine tune the pre-trained model. Besides, subsequent to the training of GNNs on selective (representative) cities, we believe that it is possible to transfer the model to new cities. On the one hand, such transfer and application of data-driven models are already practiced in the field of strategic transport modelling, e.g. the transfer of mode choice models from one city to the other [49]. On the other hand, the possibility of building such generic data-driven models are also shown in the literature (e.g. [50]). Fine-tuning can then be applied to additionally make the GNN model even more accurate for the distinct local network conditions, if any. Samples required for fine-tuning are already present today, e.g.

ALGORITHM 1 Abstract surrogate model generation algorithm with fine tuning.

Require: $f_{4\text{SM}}, \mathbb{G}(f_{4\text{SM}}), \mathbb{G}_n, \mathcal{D}$

$D_{\text{augmented}} \leftarrow \emptyset$

while $|D_{\text{augmented}}| \leq \mathbb{G}_n$ **do** ▷ 1. Generate augmented dataset

$D_{\text{augmented}} \leftarrow D_{\text{augmented}} \cup \{\mathbb{G}(f_{4\text{SM}})\}$

end while

$g \leftarrow t(\emptyset, \mathcal{G}_{\text{GNN}}, D_{\text{augmented}})$ ▷ 2. Train GNN on augmented data

$g \leftarrow t(g, \mathcal{G}_{\text{GNN}}, \mathcal{D})$ ▷ 3. Fine tune g on observed data, if the dataset exists

return g

data from traffic sensors. Furthermore, on the one hand, these data are increasingly available with digitization, while on the other hand, emerging automatic model calibration algorithms (e.g. [51, 52]) enables the possibility of having a streamlined process through the application of MLOps. This type of pattern learning using larger datasets and then fine-tuning a pre-trained GNN model with smaller samples are shown to be successful and effective in the transportation field [53] as well as in other fields [54].

3.3 | Abstract algorithm for the surrogate model

The abstract algorithm to produce a surrogate model is defined in Algorithm 1. This is the entire procedure of the abstract algorithm, including data generation, training on the generated data and fine tuning on the observed data (e.g. traffic counts, in case it exists).

The algorithm takes as input a calibrated four step model ($f_{4\text{SM}}$), a generator ($\mathbb{G}(f_{4\text{SM}})$), the number of augmented datasets to be created ($\mathbb{G}_n$) and true observations ($\mathcal{D}$), if available. The algorithm's output is a surrogate function g , which was initially trained to mimic the calibrated four step model, and finally, is fine-tuned on true, observed data. If there is no true data provided, the algorithm returns a surrogate for the four step model without fine tuning. In the current paper, only the calibrated four step model is investigated.

Looking at how the algorithm works, the first step is to generate the amount of augmented samples required, by calling the $G(f_{4\text{SM}})$ function and saving the output to the $D_{\text{augmented}}$ dataset. With every call, the generator creates random socio-economic and network data, and returns the traffic output of the four step model. Once the augmented dataset is created, the next step is to train a graph neural network on the augmented dataset, by calling the $t(\emptyset, \mathcal{G}_{\text{GNN}}, D_{\text{augmented}})$ function. The empty set denotes that there are no initial models, $\mathcal{G}_{\text{GNN}}$ shows that that the surrogate should be from the class of graph neural networks, and the $D_{\text{augmented}}$ is the training data. Finally, after training on the augmented data, the model can be fine-tuned on true data with the final t training function call, using the pre-trained model g as the initial configuration.

3.4 | Models estimated

This section briefly describes the baseline and the GNN models that are estimated in this paper.

3.4.1 | Baseline models

By estimating multiple baselines, it is possible to ascertain both how hard a problem is, as well as validate the improvements shown by the more complex models. The baseline models are those that are used often in machine learning on a wide variety of tasks. Additionally, baseline models have implementations in existing libraries, making them easy to apply to new data settings.

This study involves both classification and regression tasks. For the classification tasks, the majority classifier predicts the output class that was most often in the training data. This can be seen as the most simple classifier, and is used as the absolute lower bound. For regression task, the lower bound model is the mean regressor, which predicts the mean of the training data for all input data. This is the simplest regressor possible, equivalent to a linear regressor which has only the intercept.

Random forest models [55] are also used as baseline models for the type of tasks. The random forest model is estimated using Scikit-Learn [56]. Besides, deep learning models, also called the multilayer perceptron (MLP), are also estimated. If not otherwise stated, the MLP contains three hidden layers, each with 100 neurons, and training is done with the Adam optimizer. Training is performed until three consecutive epochs have a loss function difference less than 10^{-4} . The MLP is also implemented using Scikit Learn.

3.4.2 | Graph neural networks

The MLP is a highly flexible model. However, it does not explicitly consider the graph structure of the transport network in its computations. The class of graph neural networks (GNNs) extend the deep learning framework to directly use graph information. In this study, GCN networks [57] was estimated. A GCN is a neural network that consists of stacked GCN layers. The input of a GCN is the ordered nodes and their features. Then, in each GCN layer, the neighbouring nodes exchange their feature vectors before performing a standard MLP step. The output of a GCN is thus also on the same graph as the input data, but with processed features. The intuition of a GCN with n layers is that any two nodes within n hops over the graph will exchange their information. Thus, the number of layers selected for the entire network also determines the maximum graph size that can be effectively used for transport planning. Regarding the training of the GCN, each operation is differentiable, meaning that the backpropagation method can be employed to calculate the gradient of all weights. Finally, stochastic gradient descent is applied to optimize the weights using the Adam optimizer.

A problem with the basic GCN (version 1) is that it suffers from the over-smoothing problem [58]. The over-smoothing problem occurs when the features of the individual nodes become highly similar to each other, as more GCN layers are stacked together. To overcome the over-smoothing issue, Chen et al. [59] propose two extensions. The first extension is a residual connection to the initial layer, meaning that a weighted average is performed for each node at every layer between its current feature and the input. The second extension is to perform a weighted average between the shared weight matrix and the identity matrix. The averaging weights, defined as α and θ respectively, are hyperparameters of the model. The result of these two extensions is that the model can have over 60 layers without performance degradation. For the formal definitions, the reader is referred to the original article. The modified model is named as GCNII. The hyperparameters are analysed and discussed in the experiments section later in the study.

Regularization:

By using differentiable functions GNNs can be trained with the backpropagation algorithm. However, the models are incredibly complex, making training often unstable or coming into sub-optimal configurations. Usually, there are more free parameters that need to be trained than there are data samples [60]. The effect often leads to neural networks overfitting on the training dataset, where they memorize the values instead of learning the underlying pattern of the data, thus performing poorly. Regularization techniques attempt to overcome this issue, and specifically two state of the art methods are used extensively in this study, namely Dropout and GraphNorm.

Dropout [61] is a simple regularization technique which randomly drops a pre-determined percentage of connections between two layers. The core idea is to force the layers to learn robust representations which are not dependent on specific outputs. Note, dropout is only applied during training between all layers, whilst during evaluation runs, the full network is used. GraphNorm [62] is a recently proposed method that improves convergence properties of many popular GNNs by standardizing the data between layers. The idea is to standardize each node's feature vector by subtracting the mean of all node's vectors, and then applying a shift by a learned coefficient. Finally, each feature vector is divided by its standard deviation. Cai et al. [62] show both improvements in convergence and generalization performance for a number of GNNs. When GraphNorm is used, it is applied after a graph operation but before the activation function. Note, GraphNorm is not always used, due to the extra computational effort and GPU memory usage. The experiments that use GraphNorm are clearly defined in the experiment setup section further on.

Loss functions and evaluation metrics:

Each neural network requires a loss function with which the gradients of the weights are calculated and finally, optimized for training. On a different note, evaluation metrics summarize the performance of the models from a specific viewpoint and quantitatively validate how well they work. All loss and evaluation functions take a model as well as a dataset as input,

whilst providing a certain score as output. However, multiple functions should be considered simultaneously to validate certain improvements, and qualitative analysis should also always be performed. The following paragraphs first look at the loss functions used and then at different evaluation metrics.

Loss functions are used to train graph neural networks, by providing a score and consequential gradient for a prediction of the model on a dataset. Each component of the loss function needs to be differentiable. Additionally, loss functions need to be numerically stable and faster to be evaluated on GPUs. The overall aim of the training is thus to create a model which minimizes the loss function. Cross entropy is used as the loss function for the classification tasks, while mean squared error (MSE) is utilized for regression tasks.

With regards to the evaluation of the classification tasks, accuracy and F1-score are investigated in this study. In the case of regression tasks for real valued predictions, mean absolute error (MAE) is used at first. As the data often contains 0 values, a modified metric is introduced, based on MAE, where only the values with targets with at least 10 veh/h are taken into consideration. Besides the MAE, R^2 is also investigated. With the same line of thought as $MAE \geq 10$, a modified R^2 version is implemented, using only target values with at least 10 veh/h.

3.5 | Assumptions & methodological scope

The following aspects are assumed:

- Networks are generated with 15 to 80 nodes, and it is assumed that they are able to represent small networks realistically, whilst the results on these networks can be generalized to larger networks.
- When extracting random subnetworks from a real large network, the subnetworks are distinct and realistic when used with the four step model.
- For the data generation, random procedural methods are able to represent real networks.
- Fine tuning on observed data is not performed.

The assumptions bring the scope of the surrogate in this paper to a proof of concept. The key assumption posed here is on the data generation scheme, but it is necessary due to the lack of data, and in future surrogate models, it can be replaced without reducing the validity of the general approach. Based on the assumptions, algorithm that will be implemented in this study is identical to Algorithm 1, however, without performing the fine tuning step.

The implementation of the entire pipeline is performed in Python. All GNNs are implemented in PyTorch geometric [63], which is an extension to the PyTorch library [64]. PyTorch contains a number of functions which enable quick development of deep learning models, such as automatic differentiation and prebuilt layers. PyTorch geometric then extends this by adding in graph specific functions, especially implementations for different varieties of GNNs, including GCNII and GATv2. Due to privacy concerns, the entire code (including the ones for data generation and processing) is provided in a private GitHub

repository, which is found at https://github.com/nikita68/4_step_model_surrogate. Access to it is possible upon request. Nevertheless, the processed datasets are separately available at <https://github.com/nikita6187/TransportPlanningDataset>.

4 | DATA GENERATION AND TRANSFORMATION PROCEDURES

A key component of the surrogate modelling paradigm is the generation of a large amount of data. A description of how the input network data is created and how the four step model is used for the surrogate training is presented in this section, including the steps of the data transformation pipeline.

An augmented dataset generation procedure, based on a real-world network with actual nodes and edges, is proposed to generate the input data for the four step model. Then, a basic four step model is utilized to determine the transport usage volumes, that are used as targets for the surrogate models. Finally, the data transformation pipeline transforms the raw data into datasets that can be used by deep learning graph neural networks. A total of 29 modules are implemented as part of the data transformation pipeline, which can be categorized as attribute processing, graph processing, PyTorch tensors and visualizations.

4.1 | Data generation

This section focuses on the data generation process, which is needed as not enough publicly accessible and standardized data exists. Strategic transport modelling approaches start with trip generation process based on socio-demographic data, either at aggregate (e.g. in the case of four-step approach) or disaggregate level (e.g. in the case of agent-based approaches). The final output is the traffic levels in various links (usually based on equilibrium mechanisms), with the generation of OD matrices (generic and mode-specific) as intermediate steps. To the best of our knowledge, there is no real-world dataset, which contains the duo of socio-demographic data (corresponding to the traffic analysis zones) and the traffic levels (in the links of a complete network) for numerous cities, as required for GNN training. Therefore, an option is to generate the augmented data and the outputs from the four-step model for having the required sample for model training. Based on [65, 66], it is assumed that 10,000 samples are required to create a surrogate for the four step model. Each sample can be considered as a proxy to a city network, containing the network and socio-economic data and the resulting transport usage. A data generation scheme is presented, together with a basic four step model that is going to be the target function for the surrogate model.

4.1.1 | Augmented data generation

To create realistic networks, sub networks are extracted from a real-world network and then synthetic zonal data are appended to it. The following procedure was inspired by previous works

in procedural generation, random walks and path finding methods [67–70]. Such a model training based on sub-networks is also observed in the transportation field (e.g. [71]). Assuming that the number of nodes (G_{nodes}) and the real-world network data is provided as input, the algorithm steps are as follows:

1. Load the network data.
2. Select a random start node.
3. Perform breadth-first search from the start node until G_{nodes} are reached. Extract all nodes and links within the search space.
4. Calculate the number of zones to be created, i.e. $G_{zones} = G_{nodes} * G_{zones/nodes}$. Select G_{zones} among G_{nodes} , which are furthest away from each other to become zonal nodes.
5. Generate zonal information (e.g. number of residents and jobs) for the previously selected nodes, i.e. the zonal nodes.
6. Run the four step model on the data, and save the data and the output of the four step model.

The zonal information generation is done at first by designating the zones that are residential focused and that are workplace focused, though mixed zones are also generated implicitly. Then the total number of workplaces is sampled from a Gaussian distribution, with a mean of 1000 workplaces per zone and a standard deviation of 100. A similar approach is followed for the sampling of residents. Then, using another Gaussian distribution, the car ownership is determined for each residential zone, with a mean of 25% having no car, and a standard deviation of 10%, based on [72]. Finally, shopping is distributed randomly uniformly over the zones.

The source network data corresponds to that of the Greater Munich metropolitan region (which includes Munich and other surrounding smaller cities) and is derived from the Open Street Map [73]. After the required initial processing, the network is filtered only to include the major links, resulting in 68,424 nodes and 138,080 links. Further simplifications are conducted by having only two road types: low capacity and high capacity. Low capacity roads have a capacity of 500 veh/h at a maximum speed of 30 km/h, whilst high capacity roads allow 2000 veh/h at a maximum speed of 50km/h, reflecting the roads in urban areas. The simplification is conducted to ease the problem for the neural network surrogate while developing the proof of concept.

The generation procedure is implemented in Python 3.8 [18], using an API to communicate with PTV Visum 2021 [74]. The aforementioned algorithm is run 10,000 times to create an appropriate amount of training data. Each generation instance is provided with a target amount of nodes to create, with the total number being uniformly distributed between 15 and 80 nodes over all instances. To maximize generation speed, multiple instances of PTV Visum are launched simultaneously. The total running time for one dataset of 10,000 samples takes around 26 h to create on a consumer level Intel i7 with 16 GB of RAM. Finally, each sample's data, assumed to represent a city, is saved as a JSON file, to be both easily human and machine-readable.

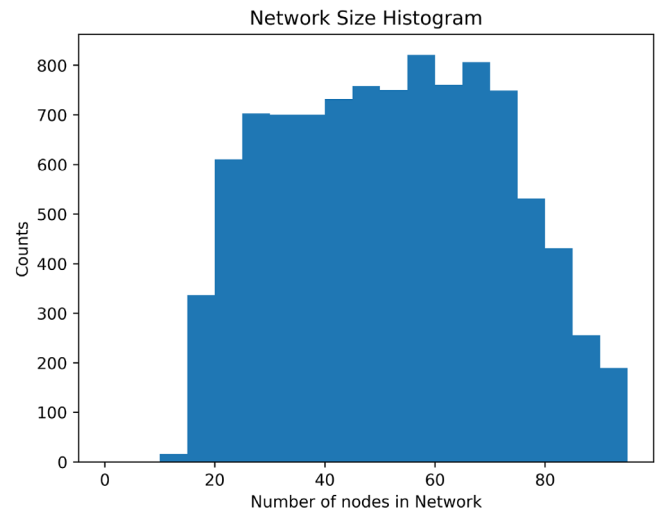


FIGURE 2 Augmented dataset histogram of graph sizes. Note, the network size is not distributed uniformly as the algorithm prioritizes realistic networks over the exact number of target nodes.

A histogram of the network sizes is provided in Figure 2. Note, this is not a perfectly uniform distribution, as the algorithm to extract networks prioritizes more realistic networks rather than extracting the exact number of nodes. The prioritization is based on using breadth-first search, adding depth until the number of nodes is within the given range. Stopping the algorithm in between depth stages is possible, but could result in networks with more dead ends. Finally, the dataset is split into 6400 samples for the training, 1600 for the validation and 2000 for the test dataset, motivated by the splitting values often used in practice in machine learning [75]. The dataset splits are kept constant across all experiments.

It is to be noted that each sample in the dataset is a graph on its own. The model training will be performed under inductive setting and as such, the unique structures that will be present in the test dataset are unknown to the trained GNN model. Therefore, the prediction scores on the test data would reflect the performance of the model for new cases.

4.1.2 | Four step model implementation

A four step model implemented in in PTV Visum 2021 is used to generate the transport usage (traffic) data. The coefficients and the setup of the four step model is held constant between each evaluation. Future work can explore using different four step model setups. However, to determine whether a basic surrogate model works, we believe that it is essential to keep the original four step model function fixed.

The setup is focused on simulating a single AM peak hour, 8 am–9 am. Three modes are assumed to be available: bus, car and walking. Walking can be combined with bus within the simulations for travellers to get to the bus stop. There are four main characteristics for each zone: number of households with a car, number of households without a car, amount of work area, amount of shopping area. For the volume delay function,

which is used in the four step model to simulate the delay experienced by travellers given a specific travel volume, a standard BPR function is used.

Regarding the four step procedure, the setup was determined based not only on the Munich network, but also based on the real models of the city of Regensburg and Halle network from PTV. At first, trip generation is performed using standard cross-classification. Next, trip distribution is applied with a gravity model. After that, the mode choice is conducted using a logit model. For private transportation, a stochastic traffic assignment step is performed. Next, a loop to the trip distribution step is applied, which is run till the private transportation assignment converges. Finally, outside the loop, the public transport is assigned using headway assignment, assuming a 20-min headway on all bus lines. All interactions with PTV Visum are done via the Python interface.

4.2 | Data transformation

In this section, key data transformation steps are described, which are essential to be able to use the generated data for the deep learning models. Deep neural networks are incredibly sensitive to input data [76]. Graph data processing is especially complex, as most graph neural networks can only process input data from nodes, not from links. Furthermore, the practical use of graph neural networks is still a niche field, and there is no publicly available dataset for a surrogate model, similar to the one generated in this paper. Thus, the requirements for the data transformation step are to allow easy experimentation with different processing steps whilst at the same time being easy to debug.

4.2.1 | Transformation pipeline

To allow quick experimentation, a modular approach is designed for the data transformation pipeline. A system containing five distinct steps is proposed, as enlisted below:

1. Data loader,
2. Data graph processing,
3. Numpy conversion,
4. Final processing,
5. Data splitting.

In the first step, the data is loaded from the previously generated JSON files (as described in the Section 4.1), and converted into a list of graphs using the Python library NetworkX [77]. By using NetworkX, it is possible to easily manipulate and view the individual graphs. From henceforth, the list of graphs, as opposed to original raw data, is denoted as the dataset. In the second step, various graph processing methods are applied (described in Section 4.2.2). A parent class is created for all graph processing steps to facilitate modularity, with a single processing function that takes a dataset as input and produces a modified dataset as output. In the third step, a converter trans-

forms the data from the NetworkX graphs into Numpy arrays. Numpy is a Python library that allows easy and efficient manipulation of arrays [78]. The Numpy arrays are needed, as all machine learning techniques take only them as input data. Additionally, this step splits input and output data. In the fourth step, normalization techniques are applied, as neural networks perform best when the data is standardized between -1 and 1 . Finally, in the fifth step, the data is split into train, validation and test datasets.

4.2.2 | Graph transformation approaches

Several modules have been implemented as part of the data transformation pipeline. A current major limitation of graph neural networks is that most approaches can only use node data as input, and more importantly, only do predictions on nodes. This is a critical problem in the context of this paper, as the aim is to predict the traffic along the links. Thus, a significant step in the data processing pipeline is to transform the graph, so that it is possible to use the link information. Within the scope of this paper, two approaches are tested, as explained in the subsequent paragraphs. These transformations are used and compared in various experiments described in Section 5.

The first approach is to use line graphs [79]. At First, for each link in the original graph, a new node is created in the transformed graph. Next, if two links are connected by a node in the original network, then the corresponding nodes of the two links in the transformed graph are also connected. Regarding feature information, each node in the transformed graph shares its information to all outgoing edges. Due to the complexity of the line graph conversion, a different approach is also tested, based on the data transformation technique from [80]. In this case, a new node is generated for each link in the transformed graph, besides the original nodes. Though this requires careful monitoring of the transformed nodes that were originally links, it is easier to interpret and visualize. Nevertheless, as nodes do not carry the same information as links, the input data has often many zeros, which could potentially cause convergence problems with some machine learning algorithms. A comparison of the two approaches is shown in Table 1.

5 | EXPERIMENTAL SETUP

To answer the primary research question, it is necessary to run experiments and gather results. Details about the conducted experiments are described in this section.

5.1 | Classification tasks (basic surrogate models)

The first task is to explore whether a basic surrogate model can be built. For this, the initial experiment focus on performing classification of the traffic level along the links. As the model needs only to predict one of a few buckets, the

TABLE 1 Comparison of the line graph and direct graph transformations.

Graph transformation	Nodes transformed	Links transformed	Interpretability	Literature
Line graph	Original nodes removed; properties propagated to adjacent links	Original links become nodes	Poor	[39, 79, 81]
Direct graph	Original nodes stay; fill irrelevant properties with 0s	Original links also become nodes; fill irrelevant properties with 0s	Easy	[80]

classification task is easier than directly trying to perform regression. Thus, it is suited better for analysing the plausibility of building a basic GNN surrogate. To use the data in a classification task, the real valued transport data needs to be inserted into a bucket, with the bucket index then being the target. The bucketing system is also inspired in part by the levels found in congestion warning systems [82]. The buckets used are [0 veh/h, 10 veh/h), [10 veh/h, 500 veh/h), [500 veh/h, ∞ veh/h). These three buckets ensure that each bucket has enough samples, with a 62%, 21%, and 17% distribution respectively on the validation dataset. The buckets correspond to no transport usage, low transport usage and medium/high transport usage.

As stated earlier, the most advanced GNNs can only take input information and produce predictions on nodes. Thus, it is critical to find out how to transform the actual data. Two approaches are attempted in this study, namely the line graph and direct graph conversion, introduced in Section 4.2.2. The input data for each transformed node is as follows, with the differences between the line and the direct graph conversion highlighted in bold:

- Whether this is a neighbour of a zone (Boolean - **line graph only**),
- Whether this is a zone node (Boolean - **direct graph only**),
- Number of employed residents with and without a car (standardized),
- Number of workplaces and shopping opportunities, in case of zone nodes (standardized),
- Whether this is a (**line graph** - neighbour of a) stop node for a bus line (Boolean),
- Whether there is a bus line going through this link (Boolean),
- Whether this is a primary or secondary road (Boolean),
- Whether this edge is the forward direction (Boolean),
- Length of the link (standardized),
- PrT capacity and maximum speed along this link (standardized).

Any properties that are irrelevant, for example number of workplaces for a link, receive the value 0. The target in this case is the traffic level along the link. The input variable setup is held constant to all experiments, based on whether a direct or line graph conversion is used. The models estimated for this task are the three baseline models: majority classifier, random forest and MLP. The primary evaluation metric is the averaged F1 score across all classes. A small random search was initially performed for determining the hyperparameters and we observed that the performance was similar across the different parameters. Hence, hyperparameter search was not implemented in the

later stages, with the assumption that the baselines that we used are relatively robust to their hyperparameters. A GCNII (line graph) model is then estimated for this task. Here, a grid search is performed to determine the best hyperparameters, across the number of hidden layers $\in (5, 10, 50)$, the size of each layer $\in (64, 256)$, the weighted averaging parameters $\alpha \in (0.1, 0.4)$ and $\theta \in (0.5, 1.5)$ parameters.

The best performing model is found to be GCNII with 10 layers, each containing 256 units, $\alpha = 0.1$, $\theta = 0.5$ and 2 feed forward layers for the final output. This GCNII model is then trained for 300 epochs over the entire training dataset. An identical model for the direct graph conversion is constructed as in the best performing line graph model, i.e. 10 GCNII layers with 256 units. The identical model setup is carried out to ensure that the models can be compared.

5.2 | Regression task

For a better surrogate model, it is essential to find out whether exact values can be predicted in a regression setting. For GNNs, this implies only that the final layer needs to be changed, specifically the activation function. However, regression is incredibly sensitive to the data processing. The following experiment setup look into a classical regression regime. Here, the output of the model as well as the target are single real valued numbers. The loss function in the regression scenario is always mean squared error. The input data is in the identical configuration as the classification tasks from the previous experiment, i.e. augmented dataset with direct graph conversion due to its stability, but without the bucketing. The same validation set is also used here as in all previous experiments. Due to the sensitivity of the model to the target data, a number of configurations are tested, based on best practices from literature [83, 84]. The various modifications are as follows:

- log applies the natural logarithm to the targets.
- masked denotes only targets with at least 10 veh/h being used.
- linear n uses linear scaling to multiply the targets with $\frac{1}{n}$. If applied together with log, it first applies linear scaling.

Modifications are in some cases combined together, which are denoted by the “+” symbol. It is to be noted that most of the targets (around 62%) are less than 10 veh/h, and this would make the machine learning models very skewed. Considering this aspect, we performed experiments by masking the sample with target less than 10 veh/h. Nevertheless, this can reduce the representativeness of the surrogate. However, to overcome

this, in practice, one could first run a classifier to get the split for the targets with values less than 10 veh/h and then use the regression model to get the exact value for the other targets. This is possible as the results of the classification experiments show that the models have good prediction accuracy for the class '<10 veh/h' (refer Section 6 for details).

The baselines are used to verify improvements associated with the GNNs. The null model in this case is the mean regressor, taking the average of the training dataset as its output. The random forest model is used in the regression configuration, whilst the MLP is trained using MSE. With regards to GCNII, the same basic configuration that is found to be optimal in the previous experiments is implemented, i.e. 70 GCNII layers, each with 512 hidden units, $\alpha = 0.1$ and $\theta = 1.5$. The models are trained for 300 epochs. Additionally, some modifications are explored, denoted in the results section (Section 6.2) with the following names.

- GraphNorm: Additional GraphNorm layer after every GCNII layer, before the activation function.
- Res- n : With n number of residual feed forward layers, as defined in [85], which applies more processing but leading to additional GPU requirements
- Dropout- n : Apply dropout with n percent of units being masked out (with $n > 25$ in combination with GraphNorm convergence was problematic).

The configurations are also combined, as can be seen in the results section. Due to a large number of different possible configurations and each training-run taking around 8 to 12 h, a truly exhaustive search was not possible. Instead, a manual search is employed, where a number of initial configurations are iteratively improved to maximize the validation score. This strategy is considered a standard practice for deep learning models [86, 87]. Within this experiment, the initial runs attempt to find the best data transformation strategy and the final runs optimize the GCNII configuration. In the preliminary runs, it was quickly found out that the presence of a large number of 0 veh/h values skew the dataset and result in poor models, when performing regression directly. Additionally, applying the MAE directly also resulted in metrics showing better performance than actually qualitatively observed. Thus, the two metrics used here are $R^2 \geq 10$ and $MAE \geq 10$ (refer to Section 3.4.2.2 for more information about these metrics).

6 | EXPERIMENTAL RESULTS

The experiments were performed sequentially, as presented in the previous section and the respective results are summarized in this section.

6.1 | Classification task (basic surrogate models)

The first task focuses on building the basic surrogate model, by aiming to classify the traffic level of each link into three

TABLE 2 F1 score for the classification task comparing line & direct graphs.

Model	F1 score - line graph	F1 score - direct graph
Majority classifier	0.25	0.25
Random forest	0.54	0.54
MLP	0.54	0.54
GCNII	0.53	0.77

buckets. The exact model details are provided in the previous section. The first experiment compares the line graph and direct graph conversions and their effect on the outputs. The models are identical, except in that they use different graph processing steps. The results of the training runs of the models are presented in Table 2. It can be seen that for the line graph conversion, the GCNII has marginally worse performance than the baselines. A brief qualitative examination of the line graph results show that no model captures the underlying patterns, and most of the F1 score comes from predicting trivial traffic buckets with no traffic. On the other hand, a GCNII with direct graph conversion achieves significantly better results than the baselines, supported by manual examinations of individual outputs.

The convergence of all GNN models based on the direct graph is observed to be stable and level out at around 300 epochs. The training loss and the validation F1 score can be seen for the GCNII model in Figure 3. If not otherwise stated, all the presented GNN models in this study have similar convergence behaviour on the training and validation data. It can be concluded that a larger number of layers and thus more trainable parameters are helpful. Additionally, it is observed that all models perform similarly between their test (prediction performed on unseen data) and validation scores (difference $\leq 1\%$), verifying both that an appropriate data splitting mechanism is employed, as well as the generalization of the models is highly probable. With regards to the model performance, the results imply that deep GNNs are a suitable candidate to further explore for a surrogate model for the four step approach. Furthermore, given that the training is performed under the inductive setting, the model performance also reflects the performance of the model for new graph structures, which also provides a basic notion of the possibility of transferring the model for new areas.

Quantitative analysis shows that the models perform adequately; however, a qualitative examination and interpretation of the outputs help to understand the context of the models. In this section, some examples are assessed, which are randomly selected outputs. In general, the model captures the supply and demand patterns and how they are assigned across the network. An important phenomenon in transport planning is that of the congested network, where travellers decide to take an alternative and often longer route to avoid congestion. The trained model is able to predict this pattern (refer to Figure A1 in Appendix). Naturally, the GNN surrogate replicates well the source model, as can be seen in Figure A2 in Appendix. Nevertheless, in some cases, it may overpredict how many travellers take the

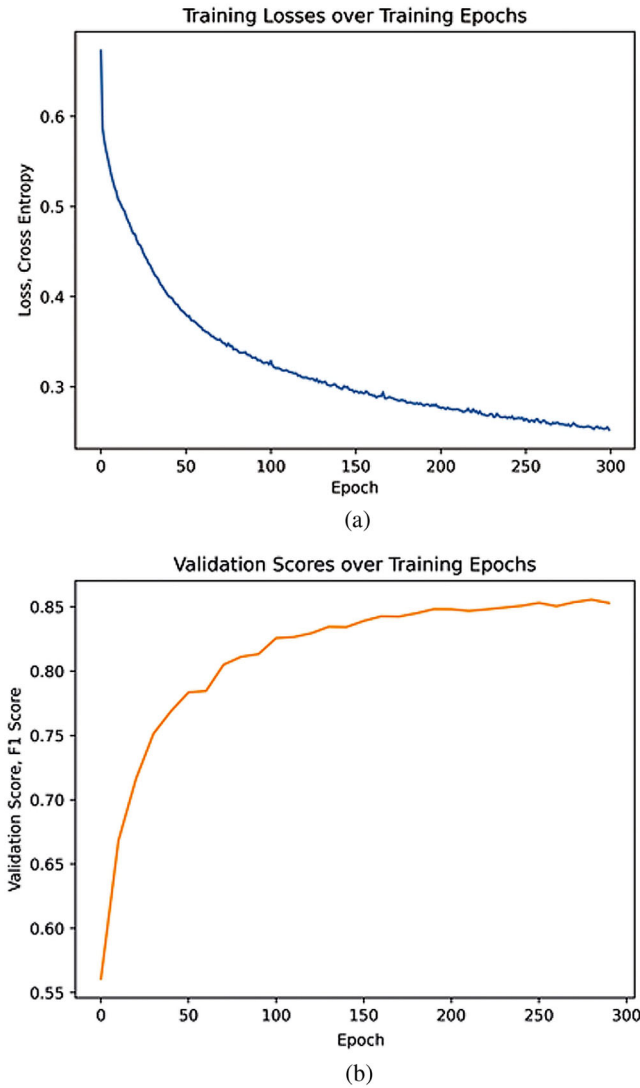


FIGURE 3 GCNII model - (a) training loss of cross entropy over the training epochs, (b) F1 score for the validation dataset.

alternate routes. Furthermore, the errors can propagate across the network, i.e. if an initial link is misclassified, then the misclassification affects the surrounding areas in the network. This effect is probably due to the way that GNNs make predictions by iteratively propagating the feature vector across nodes.

An issue with the well-established strategic transport modelling approaches is that it has long run times for large models. The run time of a four step model in Visum, for a larger network from the current dataset, is around 15 s, whilst it takes around 0.3 s for a trained GNN model. The result translates into a 50 time speed up and this is without any use of GPUs. Further speed up can easily be achieved by using GPUs and batching predictions performed together. Furthermore, the GNN-based surrogate also outputs the probabilities for the predictions (e.g. Figure 4). Summarising the findings, the answer to whether a basic transport planning surrogate model can be built is yes. The models perform well for the class of $[0, 10)$ veh/h (F1-score of 0.95), and provide good predictions for the other two classes,

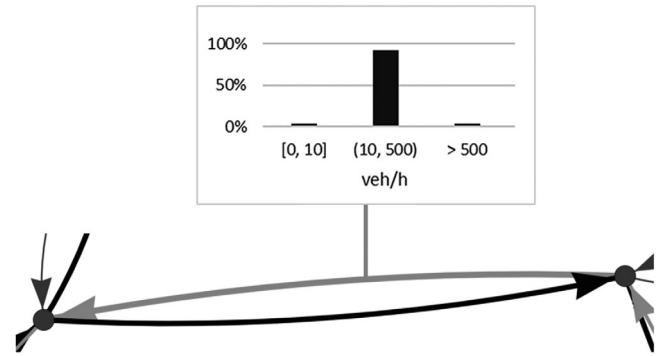


FIGURE 4 An example of probabilities of prediction for a link.

while being able to comprehend different traffic patterns. It is found that direct graph conversion is suitable to transform the graph, and deep GNNs perform better than both their shallow counterparts and other machine learning baselines.

6.2 | Regression task

Having observed that a classification setting works, it is important to find out whether exact values can be predicted in a regression setting. The regression experiments are based on the setups described in Section 5.2. The evaluation metrics of the estimated models can be found in Table 3. Focusing on the classical regression, the baselines perform poorly, with little improvement observed for the random forest and MLP over the mean regressor. For the GCNII model, it can be seen that applying logarithmic transformations on the targets improves the performance, but not significantly. Masking targets also improves the performance slightly. When applying a linear transformation before the logarithm function, the performance improves to around 200 MAE ≥ 10 . However, more significant performance improvement is experienced by only using a linear transformation, reaching 176.6 MAE ≥ 10 . The best performing model uses linear transformation, target masking, GraphNorm, a dropout of 25% and 3 residual final layers, and reach a MAE ≥ 10 of 131.1 and $R^2 \geq 10$ of 0.84. It was found that more residual final layers degrade the performance. With regards to the prediction times, a value similar to the one obtained for the classification task (i.e. around 0.3 s) is observed for a larger network from the current dataset.

To sum up, the regression experiments show that the linear transformation of data is best for strategic traffic prediction. Additionally, target masking, GraphNorm, dropout and residual final layers make significant improvements. However, when looking at the scores of the best model, it still has an error of around 131 (translates to 23.8%, when compared to the average target value in the validation dataset). Looking at the results of the regression setup, it can be seen that GNNs perform significantly better than the baselines. A number of hypotheses are open as to how regression performance can be improved in future research: (i) potentially more training data could help model convergence, (ii) deeper models on better hardware could

TABLE 3 Evaluation metrics for the regression tasks.

Model	$R^2 \geq 10$	MAE ≥ 10
Mean regressor	0.00	394.8
Random forest	0.00	373.1
MLP	0.00	371.6
GCNII + log	0.32	272.6
GCNII + log + masked	0.32	267.7
GCNII + linear - 250 + masked	0.66	176.9
GCNII + linear - 100 + log + masked	0.59	201.7
GCNII + linear - 500 + log + masked	0.69	197.1
GCNII + linear - 250 + masked + GraphNorm	0.80	147.9
GCNII + linear - 250 + masked + GraphNorm + dropout - 25	0.81	145.7
GCNII + linear - 250 + masked + GraphNorm + dropout - 25 + Res - 3	0.84	131.1
GCNII + linear - 250 + masked + GraphNorm + dropout - 25 + Res - 6	0.83	137.6

log, masked, linear - n denote data transformations, whilst GraphNorm, Res - n and Dropout - n represent different model configurations. When the individual configurations are combined, they are represented with the “+” symbol. Refer to Section 5.2 for more information.

potentially find more subtle patterns and (iii) in complex training settings like here, MSE might result in poor gradients for the model [88] and therefore, alternative loss functions could be explored in future.

7 | DISCUSSION AND CONCLUSIONS

There exists a significant overlap between (i) the issues of the existing transport planning methods (e.g. the sequential assumptions of the travellers’ decision making, the manual calibration process and the long run times) and (ii) the potential of a GNN-based surrogates. Therefore, this study attempts to explore whether a GNN surrogate function can be developed for strategic transport planning. As an initial step towards this direction, this paper focuses on the development of a GNN surrogate for the four step strategic transport modelling approach and examine where the limits currently exist. A formal definition of the problem is presented, and the surrogate model baselines based on the classical machine learning methods and the GCN based surrogates are implemented. For the experiments, the metrics F1-score, MAE ≥ 10 and $R^2 \geq 10$ are defined.

Augmented data generation procedure is introduced for required dataset creation. To the best of the authors’ knowledge, the augmented dataset proposed and generated in this study is the largest GNN surrogate modelling dataset currently available. Specifically, the dataset is one of the largest for GNN training for node classification or node regression. The augmented data generation is found to work well, producing converging models that can generalize their outputs to new samples, as expected from the existing literature [89]. The general problem formulation in the inductive setting, which combines GNN transfer learning together with the node classification for entire graphs, has not been researched much in the existing literature before [65]. The results of this study find that GNNs can be used for these tasks.

Based on the preliminary experiments, a stable graph processing strategy is found, which first transforms the edges into links. It is demonstrated that a basic surrogate model can be built, with consistent convergence and the ability to model critical transport aspects, such as the congested network. On the classification scenario, the models achieve high prediction accuracy. Based on existing works, the classification models reach the accuracy required for section control and congestion warning systems [82]. Furthermore, when comparing the run times, the GNN models have a 50 time speed up than the PTV Visum baseline and provide confidence intervals for the outputs. This is in line with similar surrogate results in other simulation fields [16, 44].

A comparison of regression and classification results shows that the latter, or more formally cross entropy, is more stable than the regression loss (i.e. MSE), with respect to the hyperparameters. This is also in line with existing literature from other fields [90, 91]. On a different note, the experimental results show that deeper GNNs perform better than their shallow counterparts, which is not been proved often in the existing literature [59], primarily due to subpar model formulations and a lack of data. To summarize, the answer to the primary research question, i.e. looking at whether a proof of concept surrogate can be built, is yes. A configuration of data generation, transformation and GNN models is determined, which results in stable convergence, based on augmented data, direct graph conversion as well as deep GNNs. Overall, the surrogate not only replicates the results of the source model, but also predicts the traffic dynamics well. Hence, the model can be deemed to be suitable for exploratory scheme assessments. While the existing strategic transport modelling approaches have their advantages, the GNN-based surrogates are better over them in some aspects, as summarized in Table 4. The GNN-based surrogate might not (yet) be a complete replacement for the existing approaches, but it is a viable alternative with its own merits.

TABLE 4 Qualitative comparison between the existing strategic transport modelling approaches and the GNN-based surrogate.

Parameter	Traditional four-step approach	Agent-based approach	GNN-based surrogate
Model creation time	Medium	High	Low ^{*,#}
Data requirements	Low	High	Medium ^{*,#}
Parallel & distributed computing	No	Partial	Yes
Consideration of non-linearity	No	Partial	Yes
Uncertainty quantification (confidence scores)	No	No	Yes
Granularity	Low	High	Low [#]
Consideration of user heterogeneity	Low	High	Low [#]
Modelling of activities & tours	No	Yes	No [#]
Extent of scenario analysis	Small	Large	Small [#]
Model runtime	Minutes to hours	Several hours to days	Seconds
Automatic model calibration & deployment	No	No	Yes
Model monitoring & CI/CD pipelines	No	No	Yes
Interpretability & explainability	High	Medium	Medium

Note:

1. *Transfer learning and augmented data generation process can be helpful.

2. #In the current proof-of-concept, the GNN-surrogate is based on the four-step approach. However, the end-goal is to surrogate an advanced agent-based approach. Therefore, the attributes associated with the GNN-surrogate might change in future.

3. Model creation time: time required to create a model for a new city.

4. CI/CD - Continuous Integration/Continuous Delivery.

5. Interpretability: the extent to which you are able to predict what is going to happen, given a change in input or algorithmic parameters.

6. Explainability: the extent to which the internal mechanics of a model can be explained.

Naturally, this study has its limitations. Moving towards data-driven models can create some apprehensions. For example, the black-box nature of the deep learning models can make it difficult to explain the decision-making processes and can be concerning. However, there are more than 10 methods for GNN model interpretability, including both local- and global-level explainability [92–94], which can support understanding of the decision-making process within the model. Another drawback with data-driven methods, such as the neural networks, is their data hungriness. While augmented data is used in this study to train the models, future studies could evaluate the impact of increasing the training data using synthetic data generators. Conversely, more real world networks can be employed, both for model training and evaluation. While the issue of over-fitting is taken care through the evaluation of the performance on the test data set, the generalizability of the models to real-world networks from multiple cities and its transferrability is yet to be verified in further studies, although the existing literature concludes that GNNs possess such properties, albeit under certain conditions. An issue with data-driven models is that they may neglect basic physical principles (e.g. road capacity constraints), which also needs further investigation.

Considering the fact that classification is found to be more stable than regression, while also taking into account that exact traffic values from a regression setting are preferable, future studies can explore the possibility of building hierarchical model (combining classification and regression tasks into a single model). With regards to the type of GNNs, this study focused on the GCNs, wherein the neighbour aggregation function used puts equal weight on all neighbours. In the subsequent research,

graph attention network could be investigated. Such a model can understand the differences between the neighbours based on the concept of attention. Given that this study provides a foundation on which more powerful and accurate surrogate models can be built in future and the fact that GNNs are still developing at a rapid pace at the time of writing this paper, the state of the art should be examined in subsequent studies.

Looking at the proof of concept for the four step approach, the corresponding model used as the target is basic in nature, e.g. using simple socio-economic data. Future work also needs to employ more realistic four-step models, which also require more data to be generated. Furthermore, more complex approaches, such as the agent-based ones, can be investigated. Advanced approaches, such as the agent- and activity-based approaches, are highly complicated for application and also have very high run time, creating an aversion in the minds of policymakers [4]. Therefore, the development of a GNN surrogate for such approaches would be more beneficial and the current proof-of-concept provides the necessary foundation. A combination of the strategic modelling approaches with the data-driven GNNs could also be a future research direction, since the current experiments proved to be successful. In fact, the extension of the four step transport modelling using other kinds of data-driven models has already been performed in [4], which is shown to be feasible.

On a different note, the surrogate model is envisioned to be fine-tuned based on direct data from the real-world (e.g. traffic counts). However, this step has not been implemented in this study and is a direction for future research. The existing literature from other fields show that GNN surrogates can

generalize to domains much larger than the training data [17]. Nevertheless, analysing this aspect on a real-world case study and investigating the value of fine-tuning to reduce the inaccuracies imbibed from surrogating a four-step model are other directions for future. The process of fine-tuning can also be investigated for the application of the developed GNN surrogates in multiple cities. It is to be noted that the term ‘transfer learning’ has a similar construct to the term ‘fine-tuning’. However, the former usually refers to the transfer of a model for a different (related) task, and not the transfer to a different place for the same problem. Hence, we use the latter term.

The study evaluated the applicability of GNN surrogates for transport planning through the development of a surrogate for the classical strategic transport planning approach, i.e. the four step approach. Nevertheless, many problems in the transport sector can be formulated as node classification tasks, and even more as general problems for graphs. GNNs are more appropriate, as many transport problems involve graphs, either real or abstract networks. Therefore, future research should examine how GNNs can solve other problems in transport planning. By taking the underlying network explicitly into consideration, higher accuracies could be achieved in comparison to traditional machine learning methods. Additionally, GNNs could potentially be used in operations research tasks to approximate the optimum with much faster convergence [95], for example for the fleet setup. Furthermore, the models presented achieve 95% accuracy for determining links with no transport usage, which could be used to optimize existing algorithms for traffic simulation.

Looking more closely at GNNs, there are multiple issues to be researched in the future. The first one is that they are shown to have performance degradation due to over-smoothing and over-squashing, when building deeper models for larger graphs. Over-smoothing has been primarily solved with the strategy associated with GCNII, but over-squashing remains a critical issue that needs to be addressed. With larger graphs, there might be concerns regarding the computational burden and the feasibility of a GNN surrogate development. However, existing literature shows the successful application of GNNs for graphs with more than million nodes, while the transport networks usually have less than 1000 zonal nodes. Sampling techniques (node-, layer-, and graph-wise) are usually applied to avoid the heavy computational burden [96]. In addition, with the application of GPU accelerators and distributed systems, GNN model can be developed even for graphs containing hundreds of millions of nodes [97]. Furthermore, it is also possible to train GNNs on smaller graphs and then transfer the trained GNNs to be executed on large-scale graphs [98]. Nevertheless, in the current case, there is a necessity to devise a solution to stop large error propagation, potentially similar to the training function of large language models in natural language processing [99]. Next, GNNs are currently using a fixed depth, meaning that the theoretical limit on the network size that they can process is directly proportional to their depth and fixed for a model. Future research should examine how to break this barrier, for example by using similar ideas as recurrent neural networks [100].

AUTHOR CONTRIBUTIONS

Santhanakrishnan Narayanan: Conceptualization; data curation; investigation; methodology; project administration; supervision; validation; visualization; writing—original draft; writing—review and editing. **Nikita Makarov:** Data curation; formal analysis; investigation; methodology; validation; visualization. **Constantinos Antoniou:** Funding acquisition; supervision; writing—review and editing.

ACKNOWLEDGEMENTS

This research is supported by Deutsche Forschungsgemeinschaft (DFG) through TUM International Graduate School of Science and Engineering (GSC 81, MO3 Project) and by European Union’s Horizon Europe programme under grant agreement No. 101076963 [project PHOEBE (Predictive Approaches for Safer Urban Environment)].

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

The raw data that support the findings of this study are available upon reasonable request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions. Nevertheless, the processed data are provided in a public GitHub repository, the link to which is included in the paper.

ORCID

Santhanakrishnan Narayanan  <https://orcid.org/0000-0003-1374-0406>

Constantinos Antoniou  <https://orcid.org/0000-0003-0203-9542>

REFERENCES

- Chetty, R., Hendren, N., et al.: The Impacts of Neighborhoods on Inter-generational Mobility: Childhood Exposure Effects and County-Level Estimates. Harvard University, Cambridge, MA (2015)
- deDios Ortúzar, J., Willumsen, L.G.: Modelling Transport. Wiley, New York (2011)
- Rasouli, S., Timmermans, H.: Uncertainty in travel demand forecasting models: literature review and research agenda. *Transp. Lett.* 4(1), 55–73 (2012). <https://doi.org/10.3328/tl.2012.04.01.55-73>
- Narayanan, S., Salanova Grau, J.M., Frederix, R., Tympakianaki, A., Masegosa, A.D., Antoniou, C.: Modeling of shared mobility services - an approach in between aggregate four-step and disaggregate agent-based approaches for strategic transport planning. *J. Intell. Transp. Syst.* 1–18 (2023)
- Narayanan, S., Chaniotakis, E., Antoniou, C.: Shared autonomous vehicle services: a comprehensive review. *Transp. Res. Part C: Emerging Technol.* 111, 255–293 (2020). <https://www.sciencedirect.com/science/article/pii/S0968090X19303493>
- Zito, P., Amato, G., Amoroso, S., Berritella, M.: The effect of advanced traveller information systems on public transport demand and its uncertainty. *Transportmetrica* 7(1), 31–43 (2011). <https://doi.org/10.1080/18128600903244727>
- Vos, J.D.: The effect of COVID-19 and subsequent social distancing on travel behavior. *Transp. Res. Interdiscip. Perspect.* 5, 100121 (2020). <https://doi.org/10.1016/j.trip.2020.100121>
- Zannat, K.E., Choudhury, C.F.: Emerging big data sources for public transport planning: a systematic review on current state of art and

- future research directions. *J. Indian Inst. Sci.* 99(4), 601–619 (2019). <https://doi.org/10.1007/s41745-019-00125-9>
9. Efron, B., Hastie, T.: *Computer Age Statistical Inference*, vol. 5. Cambridge University Press, Cambridge (2016)
 10. Iliashenko, O., Iliashenko, V., Lukiyanenko, A.: Big data in transport modelling and planning. *Transp. Res. Proc.* 54, 900–908 (2021). <https://doi.org/10.1016/j.trpro.2021.02.145>
 11. Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M.P., et al.: A survey on deep learning. *ACM Comput. Surv.* 51(5), 1–36 (2019). <https://doi.org/10.1145/3234150>
 12. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S.: A comprehensive survey on graph neural networks. *IEEE Trans. Neural Networks Learn. Syst.* 32(1), 4–24 (2021)
 13. Rahmani, S., Baghbani, A., Bouguila, N., Patterson, Z.: Graph neural networks for intelligent transportation systems: a survey. *IEEE Trans. Intell. Transp. Syst.* 24(8), 8846–8885 (2023). <https://doi.org/10.1109/TITS.2023.3257759>
 14. Sudret, B., Marelli, S., Wiart, J.: Surrogate models for uncertainty quantification: an overview. In: 2017 11th European Conference on Antennas and Propagation (EUCAP), pp. 793–797. IEEE, Piscataway, NJ (2017)
 15. Sun, L., Gao, H., Pan, S., Wang, J.X.: Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data. *Comput. Methods Appl. Mech. Eng.* 361, 112732 (2020)
 16. Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., Battaglia, P.: Learning mesh-based simulation with graph networks. *arXiv:2010.03409* (2020)
 17. Choi, Y., Kumar, K.: Graph neural network-based surrogate model for granular flows. *Comput. Geotech.* 166, 106015 (2024). <https://doi.org/10.1016/j.compgeo.2023.106015>
 18. Van Rossum, G., Drake, F.L.: *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA (2009)
 19. Yuan, H., Zhang, H., Zhang, J., Huang, Y., Yu, T., Zhang, H., et al.: *Deep Reinforcement Learning: Fundamentals, Research and Applications*. Springer, Singapore (2020). <https://doi.org/10.1007/978-981-15-4095-0>
 20. Li, Y.: Deep reinforcement learning: an overview. *arXiv:1701.07274* (2017)
 21. Narayanan, S., Antoniou, C.: Electric cargo cycles - a comprehensive review. *Transp. Policy* 116, 278–303 (2022)
 22. Saleh, S.M., Lulusi, L., Apriandy, F., Fisiani, J., Salmannur, A., Faisal, R.: Trip generation and attraction model and forecasting using machine learning methods. *IOP Conf. Ser.: Mater. Sci. Eng.* 1087(1), 012021 (2021). <https://doi.org/10.1088/1757-899x/1087/1/012021>
 23. Xiao, T., Lu, H., Sun, Z., Wang, J.: Trip generation prediction based on the convolutional neural network-multidimensional long-short term memory neural network model at grid cell scale. *IEEE Access* 9, 79051–79059 (2021). <https://doi.org/10.1109/access.2021.3083683>
 24. Mukherjee, J., Kadali, B.R.: A comprehensive review of trip generation models based on land use characteristics. *Transp. Res. Part D: Transp. Environ.* 109, 103340 (2022)
 25. Pourebrahim, N., Sultana, S., Niakanlahiji, A., Thill, J.C.: Trip distribution modeling with twitter data. *Comput. Environ. Urban Syst.* 77, 101354 (2019). <https://doi.org/10.1016/j.compenvurbysys.2019.101354>
 26. Goel, S., Sinha, A.K.: ANFIS based trip distribution model for delhi urban area. In: 2015 2nd International Conference on Computing for Sustainable Global Development (INDIACom), pp. 453–457. IEEE, Piscataway, NJ (2015)
 27. Pitombo, C.S., deSouza, A.D., Lindner, A.: Comparing decision tree algorithms to estimate intercity trip distribution. *Transp. Res. Part C: Emerging Technol.* 77, 16–32 (2017). <https://doi.org/10.1016/j.trc.2017.01.009>
 28. Pineda-Jaramillo, J.D.: A review of machine learning (ML) algorithms used for modeling travel mode choice. *DYNA* 86(211), 32–41 (2019). <https://doi.org/10.15446/dyna.v86n211.79743>
 29. García-García, J.C., García-Ródenas, R., López-Gómez, J.A., Martín-Baos, J.Á.: A comparative study of machine learning, deep neural networks and random utility maximization models for travel mode choice modelling. *Transp. Res. Proc.* 62, 374–382 (2022)
 30. Zhao, X., Yan, X., Yu, A., Hentenryck, P.V.: Prediction and behavioral analysis of travel mode choice: a comparison of machine learning and logit models. *Travel Behav. Soc.* 20, 22–35 (2020). <https://doi.org/10.1016/j.tbs.2020.02.003>
 31. Narayanan, S., Makarov, N., Magkos, E., Salanova Grau, J.M., Aifadopolou, G., Antoniou, C.: Can bike-sharing reduce car use in Alexandroupolis? an exploration through the comparison of discrete choice and machine learning models. *Smart Cities* 6(3), 1239–1253 (2023)
 32. Grunitzki, R., Bazzan, A.L.C.: Comparing two multiagent reinforcement learning approaches for the traffic assignment problem. In: 2017 Brazilian Conference on Intelligent Systems (BRACIS), pp. 139–144. IEEE, Piscataway, NJ (2017)
 33. Zhou, B., Song, Q., Zhao, Z., Liu, T.: A reinforcement learning scheme for the equilibrium of the in-vehicle route choice problem based on congestion game. *Appl. Math. Comput.* 371, 124895 (2020). <https://doi.org/10.1016/j.amc.2019.124895>
 34. Bahar, P., Makarov, N., Zeyer, A., Schlüter, R., Ney, H.: Exploring a zero-order direct HMM based on latent attention for automatic speech recognition. In: ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 7854–7858. IEEE, Piscataway, NJ (2020)
 35. Manibardo, E.L., Laña, I., Del Ser, J.: Deep learning for road traffic forecasting: Does it make a difference? *IEEE Trans. Intell. Transp. Syst.* 23(7), 6164–6188 (2021)
 36. Jiang, W., Luo, J.: Graph neural network for traffic forecasting: a survey. *arXiv:2101.11174* (2021)
 37. Chang, L., Ma, C., Sun, K., Qu, Z., Ren, C.: Enhanced road information representation in graph recurrent network for traffic speed prediction. *IET Intell. Transp. Syst.* 17(7), 1434–1453 (2023)
 38. Ma, Q., Sun, W., Gao, J., Ma, P., Shi, M.: Spatio-temporal adaptive graph convolutional networks for traffic flow forecasting. *IET Intell. Transp. Syst.* 17(4), 691–703 (2023)
 39. Xiong, X., Ozbay, K., Jin, L., Feng, C.: Dynamic origin-destination matrix prediction with line graph neural networks and Kalman filter. *Transp. Res. Rec.* 2674(8), 491–503 (2020)
 40. Shan, S., Wang, G.G.: Survey of modeling and optimization strategies to solve high-dimensional design problems with computationally-expensive black-box functions. *Struct. Multidiscip. Optim.* 41(2), 219–241 (2009). <https://doi.org/10.1007/s00158-009-0420-2>
 41. Tripathy, R.K., Bilionis, I.: Deep UQ: learning deep neural network surrogate models for high dimensional uncertainty quantification. *J. Comput. Phys.* 375, 565–588 (2018). <https://doi.org/10.1016/j.jcp.2018.08.036>
 42. Allotey, J., Butler, K.T., Thiyyagalingam, J.: Entropy-based active learning of graph neural network surrogate models for materials properties. *J. Chem. Phys.* 155(17), 174116 (2021)
 43. Franco, N.R., Fresca, S., Tombari, F., Manzoni, A.: Deep learning-based surrogate models for parametrized PDEs: handling geometric variability through graph neural networks. *arXiv:2308.01602* (2023)
 44. Ogoke, F., Meidani, K., Hashemi, A., Farimani, A.B.: Graph convolutional networks applied to unstructured flow field data. *Mach. Learn. Sci. Technol.* 2(4), 045020 (2021). <https://doi.org/10.1088/2632-2153/ac1fc9>
 45. He, X., Zhao, K., Chu, X.: AutoML: a survey of the state-of-the-art. *Knowl.-Based Syst.* 212, 106622 (2021)
 46. D'Inverno, G.A., Bianchini, M., Sampoli, M.L., Scarselli, F.: On the approximation capability of GNNs in node classification/regression tasks. *arXiv:2106.08992* (2021)
 47. Das, K., Jiang, J., Rao, J.: Mean squared error of empirical predictor. *Ann. Stat.* 32(2), 818–840 (2004)
 48. Friedman, J.H.: *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, Cham (2017)
 49. Narayanan, S., Arango, J.P.R., Frederix, R., Tympakianaki, A., Antoniou, C.: Can emerging mobility solutions complement public transport and lead to a sustainable future?: a case study on Regensburg, Germany (2022). https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4219590
 50. Narayanan, S., Martinez, S.Á.O., Antoniou, C.: Household car-ownership in a world of constant change - the continued influence of traditional

- variables and the rising influence of emerging mobility scenarios. *Transportation* (2024)
51. Cao, K., You, J., Liu, J., Leskovec, J.: Autotransfer: AutoML with knowledge transfer – an application to graph neural networks. *arXiv:2303.07669* (2023)
 52. Zhou, K., Liu, Z., Duan, K., Hu, X.: Graph neural network: AutoML. In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) *Graph Neural Networks: Foundations, Frontiers, and Applications*, pp. 371–389. Singapore: Springer, Singapore (2022)
 53. Sadid, H., Antoniou, C.: Dynamic spatio-temporal graph neural network for surrounding-aware trajectory prediction of autonomous vehicles. *IEEE Trans. Intell. Veh.* (2024). <https://doi.org/10.1109/ITIV.2024.3406507>
 54. Barwey, S., Maulik, R.: Interpretable fine-tuning for graph neural network surrogate models. *arXiv:2311.07548* (2023)
 55. Ho, T.K.: Random decision forests. In: *Proceedings of 3rd International Conference on Document Analysis and Recognition*, vol. 1, pp. 278–282. IEEE, Piscataway, NJ (1995)
 56. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al.: Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12, 2825–2830 (2011)
 57. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907* (2016)
 58. Li, Q., Han, Z., Wu, X.M.: Deeper insights into graph convolutional networks for semi-supervised learning. In: *Thirty-Second AAAI Conference on Artificial Intelligence*, pp. 3538–3545. ACM, New York (2018)
 59. Chen, M., Wei, Z., Huang, Z., Ding, B., Li, Y.: Simple and deep graph convolutional networks. In: *International Conference on Machine Learning*, pp. 1725–1735. ACM, New York (2020)
 60. Belkin, M., Hsu, D., Ma, S., Mandal, S.: Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proc. Natl. Acad. Sci. U. S. A.* 116(32), 15849–15854 (2019)
 61. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* 15(1), 1929–1958 (2014)
 62. Cai, T., Luo, S., Xu, K., He, D., Liu, T.Y., Wang, L.: GraphNorm: a principled approach to accelerating graph neural network training. In: *International Conference on Machine Learning*, pp. 1204–1215. Microtome Publishing, Brookline, MA (2021)
 63. Fey, M., Lenssen, J.E.: Fast graph representation learning with PyTorch Geometric. In: *ICLR Workshop on Representation Learning on Graphs and Manifolds* (2019)
 64. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al.: Pytorch: an imperative style, high-performance deep learning library. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., Garnett, R., (eds.) *Advances in Neural Information Processing Systems*, pp. 8024–8035. Curran Associates, Red Hook, NY (2019)
 65. Dwivedi, V.P., Joshi, C.K., Laurent, T., Bengio, Y., Bresson, X.: Benchmarking graph neural networks. *arXiv:2003.00982* (2020)
 66. Errica, F., Podda, M., Bacciu, D., Micheli, A.: A fair comparison of graph neural networks for graph classification. *arXiv:1912.09893* (2020)
 67. Moore, E.F.: The shortest path through a maze. In: *Proceedings of an International Symposium on the Switching Theory*, pp. 285–292. Cambridge University, Cambridge, MA (1959)
 68. Parish, Y.I.H., Müller, P.: Procedural modeling of cities. In: *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques - SIGGRAPH '01*, pp. 301–308. ACM Press, New York (2001)
 69. Kelvin, L.Z., Anand, B.: Procedural generation of roads with conditional generative adversarial networks. In: *2020 IEEE Sixth International Conference on Multimedia Big Data (BigMM)*, IEEE, Piscataway, NJ (2020)
 70. Bai, J., Ren, Y., Zhang, J.: Ripple walk training: a subgraph-based training framework for large and deep graph neural network. In: *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. IEEE, Piscataway, NJ (2021)
 71. Tran, T., He, D., Kim, J., Hickman, M.: MSGNN: a multi-structured graph neural network model for real-time incident prediction in large traffic networks. *Transp. Res. Part C: Emerg. Technol.* 156, 104354 (2023)
 72. Kunst, A.: Car ownership in Germany 2021. <https://www.statista.com/forecasts/998697/car-ownership-in-germany> (2021). Accessed 25 Oct 2023
 73. OpenStreetMap contributors: planet dump. <https://www.openstreetmap.org> (2017). Accessed 25 Oct 2023
 74. PTV Group: Visum. <https://www.ptvgroup.com/en/solutions/products/ptv-visum/> (2021). Accessed 25 Oct 2023
 75. Reitermanova, Z.: Data splitting. In: *Proceedings of the 19th Annual Conference of Doctoral Students - WDS 2010*, vol. 10, pp. 31–36. MATFYZPRESS, Prague (2010)
 76. Sola, J., Sevilla, J.: Importance of input data normalization for the application of neural networks to complex industrial problems. *IEEE Trans. Nucl. Sci.* 44(3), 1464–1468 (1997). <https://doi.org/10.1109/23.589532>
 77. Hagberg, A., Swart, P., Chult, D.S.: Exploring Network Structure, Dynamics, and Function Using NetworkX. Los Alamos National Laboratory (LANL), Los Alamos, NM (2008)
 78. Harris, C.R., Millman, K.J., van derWalt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., et al.: Array programming with NumPy. *Nature* 585(7825), 357–362 (2020). <https://doi.org/10.1038/s41586-020-2649-2>
 79. Harary, F., Norman, R.Z.: Some properties of line digraphs. *Rend. Circolo Mat. Palermo* 9(2), 161–168 (1960). <https://doi.org/10.1007/bf02854581>
 80. Zhang, Q., Jin, Q., Chang, J., Xiang, S., Pan, C.: Kernel-weighted graph convolutional network: a deep learning approach for traffic forecasting. In: *2018 24th International Conference on Pattern Recognition (ICPR)*, pp. 1018–1023. IEEE, Piscataway, NJ (2018)
 81. Ramadan, A., Elbery, A., Zorba, N., Hassanein, H.S.: Traffic forecasting using temporal line graph convolutional network: case study. In: *ICC 2020-2020 IEEE International Conference on Communications (ICC)*, pp. 1–6. IEEE, Piscataway, NJ (2020)
 82. Gerstenberger, M., Hösch, M., Listl, G.: Überarbeitung und Aktualisierung des merkblattes für die Ausstattung von Verkehrsrechner- und unterzentralen (MARZ 1999). Bundesanstalt für Straßenwesen (BASt). http://bast.opus.hbz-nrw.de/volltexte/2018/1986/pdf/V308_barrierefreies_ELBA_PDF.pdf (2018). Accessed 25 Oct 2023
 83. Benoit, K.: Linear regression models with logarithmic transformations. pp. 23–36. London School of Economics, London (2011)
 84. Vashisht, R.: Machine learning: when to perform a feature scaling? <https://www.atoti.io/articles/when-to-perform-a-feature-scaling/> (2021). Accessed 25 Oct 2023
 85. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. IEEE, Piscataway, NJ (2016)
 86. Bergstra, J., Bengio, Y.: Random search for hyper-parameter optimization. *J. Mach. Learn. Res.* 13(2), 281–305 (2012)
 87. Yang, L., Shami, A.: On hyperparameter optimization of machine learning algorithms: theory and practice. *Neurocomputing* 415, 295–316 (2020)
 88. Zhang, Z., Sabuncu, M.R.: Generalized cross entropy loss for training deep neural networks with noisy labels. In: *32nd Conference on Neural Information Processing Systems (NeurIPS)*, pp. 8792–8802. ACM, New York (2018)
 89. Nikolenko, S.I.: Synthetic data for deep learning. *arXiv:1909.11512* (2019)
 90. Rady, H.: Classification of multilayer neural networks using cross entropy and mean square errors. *J. Adv. Comput. Sci.* 2(1), 29–48 (2008)
 91. Salman, R., Kecman, V.: Regression as classification. In: *2012 Proceedings of IEEE Southeastcon*, pp. 1–6. IEEE, Piscataway, NJ (2012)
 92. Liu, N., Feng, Q., Hu, X.: Interpretability in graph neural networks. In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) *Graph Neural Networks: Foundations, Frontiers, and Applications*, pp. 121–147. Springer, Singapore (2022)

93. Bonabi Mobaraki, E., Khan, A.: A demonstration of interpretability methods for graph neural networks. In: Proceedings of the 6th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA). GRADES & NDA '23, pp. 1–5. ACM, New York (2023)
94. Xuanyuan, H., Barbiero, P., Georgiev, D., Magister, L.C., Liò, P.: Global concept-based interpretability for graph neural networks via neuron analysis. *Proc. AAAI Conf. Artif. Intell.* 37(9), 10675–10683 (2023). <https://doi.org/10.1609/aaai.v37i9.26267>
95. Cappart, Q., Chételat, D., Khalil, E., Lodi, A., Morris, C., Veličković, P.: Combinatorial optimization and reasoning with graph neural networks. *arXiv:210209544* (2021)
96. Ma, H., Rong, Y., Huang, J.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) *Graph neural networks: scalability*, pp. 99–119. Springer Nature, Singapore (2022)
97. Wan, X., Xu, K., Liao, X., Jin, Y., Chen, K., Jin, X.: Scalable and efficient full-graph GNN training for large graphs. *Proc. ACM Manage. Data* 1(2), 1–23 (2023). <https://doi.org/10.1145/3589288>
98. Ruiz, L., Chamon, L.F.O., Ribeiro, A.: Transferable graph neural networks on large-scale stochastic graphs. In: 2021 55th Asilomar Conference on Signals, Systems, and Computers, pp. 1563–1567. IEEE, Piscataway, NJ (2021)
99. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al.: Language models are few-shot learners. *arXiv:200514165* (2020)
100. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* 9(8), 1735–1780 (1997)

How to cite this article: Narayanan, S., Makarov, N., Antoniou, C.: Graph neural networks as strategic transport modelling alternative - A proof of concept for a surrogate. *IET Intell. Transp. Syst.* 18, 2059–2077 (2024). <https://doi.org/10.1049/itr2.12551>

APPENDIX

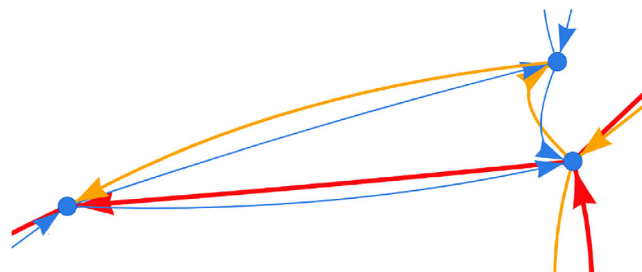
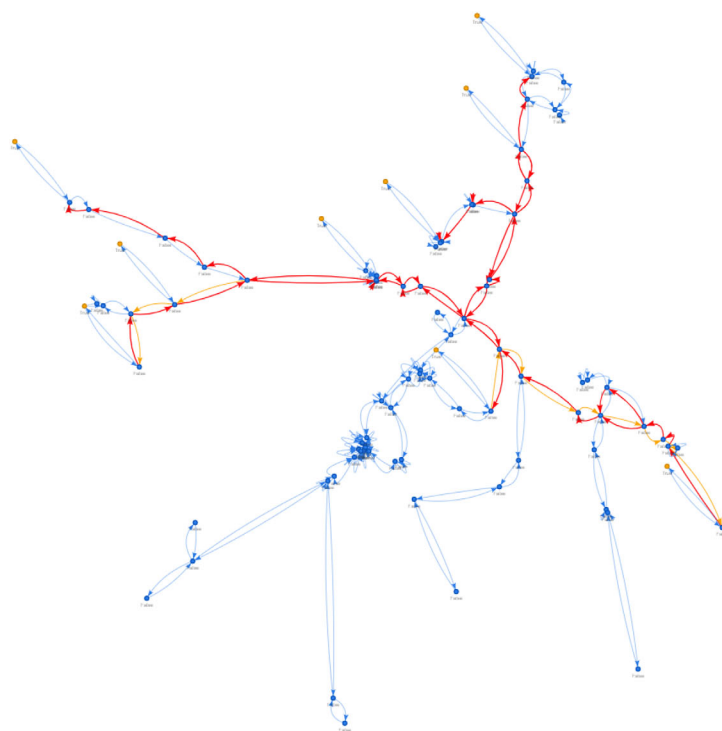
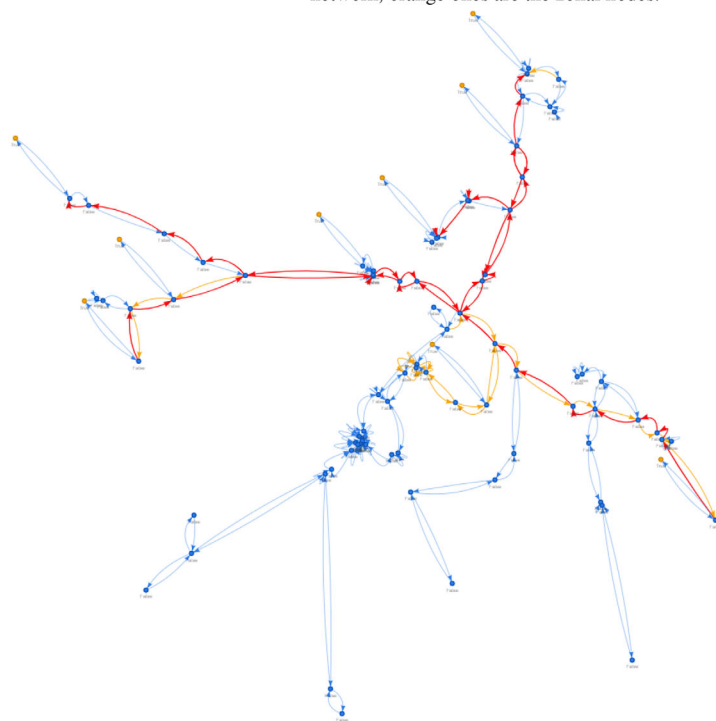


FIGURE A1 The surrogate model being able to predict the dispersion of traffic to an alternate route between two nodes. The colour of the links is as follows: blue is $[0, 10)$ veh/h, orange is $[10, 500)$ veh/h and red is over 500 veh/h.



(a) Target (based on Visum outputs) for a sample from the test dataset. The colour of the links is as follows: blue is $[0, 10)$ veh/h, orange is $[10, 500)$ veh/h and red is over 500 veh/h. While Blue nodes are normal nodes in the network, orange ones are the zonal nodes.



(b) Prediction for a sample from the test dataset. The colour of the links is as follows: blue is $[0, 10)$ veh/h, orange is $[10, 500)$ veh/h and red is over 500 veh/h. While Blue nodes are normal nodes in the network, orange ones are the zonal nodes.

FIGURE A2 A sample from the test dataset.